Access Control - One Class Period

2026-11-04

Table of contents

1 Introduction **2**

2 Before Class **2**
- 2.1 Groups and Roles . . . 2
- 2.2 Network Setup . . . 3
- 2.3 Installations . . . 4
- 2.4 A Lab Login on the Server . . . 4
- 2.5 Command Reference . . . 4

3 Concepts **5**
- 3.1 Access Control . . . 5
- 3.2 Access Control Models . . . 5
- 3.3 Ports and Port Knocking . . . 6
- 3.4 Single Packet Authorization . . . 6
- 3.5 Chroot Jails . . . 7

4 Lab: Single Packet Authorization **7**
- 4.1 Client Setup . . . 7
- 4.2 Server Setup . . . 8
- 4.3 Hiding SSH Behind SPA . . . 9
- 4.4 SPA Clean Up . . . 11

5 Lab: A Simple Chroot Jail **12**

6 Instructor Demo and Wrap-Up **14**

1 Introduction

This document covers access control in a single class period. You will learn the core ideas, then put two of them into practice:

1. **Single Packet Authorization (SPA)**: hiding a service (SSH) behind the firewall so that only a client holding the right keys can open it.
2. **Chroot jails**: confining a process so that it cannot see any files outside of a chosen directory.

The **Before Class** section must be completed before you arrive. Class time is only long enough for the steps that follow it.

Time	Activity
0-12 min	Concepts (Section 3)
12-50 min	SPA lab (Section 4)
50-53 min	SPA clean up
53-65 min	Chroot jail lab (Section 5)
65-75 min	Instructor demo and wrap-up

2 Before Class

Complete everything in this section before class. At the start of class, the server in each group should be able to show that `sudo systemctl status fwknop-server` runs without a "could not be found" error, and the client should be able to run `fwknop --version`.

2.1 Groups and Roles

Forming groups. Work in groups of three if possible: one server, one client, and one "outsider" machine. The outsider is only needed for a single step, where you confirm that the server refuses machines other than the client. If you are in a pair, team up with a neighboring pair for that step, or run a third virtual machine.

Working alone. If you do not have a partner, run both the client and the server as virtual machines on your own computer, connected with a NAT Network (see **If Bridged Networking Does Not Work** below).

Choosing roles. Each role installs different software:

Role	Installs	Needs
Server	`fwknop-server`, `openssh-server`	`sudo` access (to edit config files and firewall rules)
Client	`fwknop-client`, `nmap`	The server's ip address and a login on the server

If you cannot decide, whoever's machine already has `openssh-server` running should be the server. In a group of three, rotate who is typing so that everyone gets hands-on time.

2.2 Network Setup

1. Check your ip address. If on campus, it should be in the form 138.47.X.X.

   ```
   ip -br addr
   ```

2. If it is not, open the virtual machine's settings in your virtualization software and make sure the network adapter is set to **Bridged Adapter** with the correct connection selected (most likely Wi-Fi). In VirtualBox: select the machine, open **Settings > Network**, and set "Attached to" to **Bridged Adapter**.

3. Within the virtual machine, open the network settings and make sure IPv4 is set to **Automatic (DHCP)**. You may need to disconnect and reconnect, or restart the virtual machine.

4. Confirm that the client and server can reach each other. From the client, run `ping -c 3` followed by the server's ip address.

> **If Bridged Networking Does Not Work**
>
> Many campus Wi-Fi networks do not allow bridged virtual machines, because the VM appears with an extra hardware (MAC) address. If your VM does not get a 138.47.X.X address, or the two machines cannot reach each other, run both the client and the server as virtual machines on the same computer and connect them with a private network instead:
>
> - In VirtualBox, open **File > Tools > Network Manager**, select the **NAT Networks** tab, and create a NAT Network. Then, in each VM's `Network` settings, set "Attached to" to **NAT Network** and select the network you created.
> - Each VM will receive an address such as 10.0.2.X instead of 138.47.X.X. Use those addresses everywhere this document says 138.47.x.x.
>
> This also keeps the lab's port scanning off the campus network.

2.3 Installations

On the client:

```
sudo apt update
sudo apt install fwknop-client nmap
```

On the server:

```
sudo apt update
sudo apt install fwknop-server openssh-server
```

Then, on the server, confirm that the SSH server is listening on port 22. The output should include a line with `LISTEN` and `*:22` (or `0.0.0.0:22`).

```
ss -atn
```

2.4 A Lab Login on the Server

The client will log in to the server with `ssh` and `scp`, so the client needs a username and password on the server. Do **not** share the password to your own account. Instead, on the server, create a temporary account for the lab:

```
sudo adduser spauser
```

Choose a simple password and give it to your partner. You can leave the other information blank. This account is deleted during the clean up at the end of class.

2.5 Command Reference

You will use the following commands in class. Read over them beforehand.

- `fwknop`: the Firewall Knock Operator (pronounced "eff - double u - nop"). On the client, `fwknop` sends an SPA packet. On the server, the `fwknopd` daemon checks each packet, adds a temporary firewall rule for a valid one, and removes the rule when the time expires.
- `iptables`: manages firewall rules. Useful flags: `-L` lists rules, `-n` shows numbers instead of names (22 instead of ssh), `-v` shows details, `-A` appends a rule to a chain, `-D` deletes a rule, `-p` sets the protocol, and `-j` sets the action to take on a match (such as `DROP`).

- `ssh`: logs in to a remote machine. `scp`: copies files to or from a remote machine over SSH (`-P` sets the port).
- `chroot`: runs a command with a different root directory.
- `ldd`: lists the shared libraries a program needs.

Recall that in bash, when setting a variable, we do not put spaces on either side of the `=` sign: `x=10` is correct, while `x = 10` is not.

3 Concepts

3.1 Access Control

Access control refers to the methods and policies used to restrict and manage who can view or use resources. It is built on four key concepts:

- **Identification** is making a claim about one's identity (*who are you?*). Example: typing a username.
- **Authentication** is confirming that claim (*are you who you say you are?*). Example: entering the password for that username. Authentication is based on something you **know** (a password), something you **have** (a security key or phone), or something you **are** (a fingerprint). Multi-factor authentication combines factors from *different* categories.
- **Authorization** is specifying the access rights an authenticated identity has to resources (*are you allowed to access this?*). Example: file permissions.
- **Accountability** is the acknowledgement and assumption of responsibility for actions. It relies on logging and auditing so that actions can be traced back to an identity.

Good access control follows the principle of **least privilege** (give every user and process only the access it needs) and **defense in depth** (layer multiple controls so the failure of one is not the failure of all).

3.2 Access Control Models

Model	Who decides access	Example
Discretionary (DAC)	The owner of each object	Linux file permissions (`chmod`)
Mandatory (MAC)	A system-wide policy that users cannot override	Clearance levels; SELinux and AppArmor
Role-Based (RBAC)	The role a user is assigned	Instructors vs. students

Model	Who decides access	Example
Attribute-Based (ABAC)	Attributes of the user, the resource, and the environment	Access only from campus during work hours

3.3 Ports and Port Knocking

A port is a numbered endpoint (0-65535) that a service listens on, such as 22 for SSH or 443 for HTTPS. Ports are the way remote users get into a machine. It would be most secure to close every port, but then we could not access the machine remotely.

A port scanner such as `nmap` reports a port as:

- **Open**: a service is listening and accepts the connection.
- **Closed**: nothing is listening, and the host replies to say so.
- **Filtered**: no reply at all, because a firewall silently dropped the probe. This reveals the least to an attacker.

Port knocking keeps a service's port closed by the firewall. A daemon watches for connection attempts to a secret *sequence* of other ports. When it sees the correct "knock" from a client, it temporarily opens the real port for that client's ip address only. However, a plain knock sequence is security through obscurity: anyone who can watch the network can see the sequence and replay it.

3.4 Single Packet Authorization

Single packet authorization (SPA) improves on port knocking:

1. The client sends a single SPA packet to the server. Its contents are encrypted and protected by an HMAC (a keyed checksum that proves the packet was not altered).
2. The server verifies the HMAC, decrypts the packet, and checks the source ip address, a timestamp, and that the packet has not been seen before.
3. If the packet is valid, the server adds a temporary firewall rule opening the requested port (e.g., SSH) for the client's ip address only.
4. If it is invalid, the server silently drops it and gives no feedback.

SPA is more secure than a knock sequence because a captured packet **cannot be replayed**, it relies on **real cryptographic keys** rather than a secret sequence, and the fwknop server **does not listen on any port** (it passively sniffs UDP port 62201), so a port scan shows nothing. SPA is an *extra* layer: once the port is open, the client must still log in to SSH.

3.5 Chroot Jails

Every Linux process has a *root directory*, which it sees as `/`. The `chroot` command changes the root directory of a process and its children to another directory, called a **chroot jail**. Inside the jail, a process cannot name any file outside of it, so everything it needs (programs, shared libraries, configuration files) must be copied into the jail. A common use is limiting what SSH or SFTP users can see on a server.

A chroot jail is **not** a complete security boundary. A process running as **root** inside a jail can escape it, so jailed users must never have root access, `sudo`, or other setuid programs inside the jail. Chroot also only isolates the file system: jailed processes still share the process list, network, and kernel. Containers (e.g., Docker) build on Linux *namespaces* to provide much stronger isolation.

4 Lab: Single Packet Authorization

Throughout this lab, each step says whether it is done **on the client** or **on the server**. Use `spauser` as the username when logging in to the server.

4.1 Client Setup

1. On the client, note the client's ip address. On the server, note the server's ip address. Share them with each other.

   ```
   ip -br addr
   ```

2. On the client, set variables for both ip addresses and the SSH port (replace the x's).

   ```
   CLIENT_IP=138.47.x.x
   SERVER_IP=138.47.x.x
   SSH_PORT=22
   ```

3. On the client, test that you can log in to the server, then disconnect with `ctrl+d`.

   ```
   ssh -p $SSH_PORT spauser@$SERVER_IP
   ```

   Answer `yes` if asked whether you want to continue connecting. If this does not work, fix it before moving on: SPA cannot help if plain SSH does not work.

4. On the client, generate the keys. This creates an encryption key and an HMAC key, and saves them along with the port to open (`-A`), the client ip (`-a`), and the server ip (`-D`) as a "stanza" in `~/.fwknoprc` named after the server's ip.

```
fwknop -A tcp/$SSH_PORT -a $CLIENT_IP -D $SERVER_IP \
    --key-gen --use-hmac --save-rc-stanza
```

You should see output similar to `[+] Wrote Rijndael and HMAC keys to rc file: /home/username/.fwknoprc`. You can view the keys with `cat ~/.fwknoprc`; look for the `KEY_BASE64` and `HMAC_KEY_BASE64` lines.

5. On the client, copy the keys to the server. Enter `spauser`'s password when prompted.

```
scp -P $SSH_PORT ~/.fwknoprc spauser@$SERVER_IP:~/copied-file.conf
```

4.2 Server Setup

1. On the server, display the copied keys. The file is in `spauser`'s home directory, so `sudo` is needed to read it from your own account.

```
sudo cat /home/spauser/copied-file.conf
```

2. On the server, open `/etc/fwknop/access.conf` with `sudo nano` or `sudo vim` and find the section containing:

```
SOURCE              ANY
KEY_BASE64          __CHANGE_ME__
HMAC_KEY_BASE64     __CHANGE_ME__
```

Replace `ANY` with the client's ip address, replace both `__CHANGE_ME__` values with the matching keys from `copied-file.conf`, and add an `OPEN_PORTS` line. It should look similar to the following, with your own values:

```
SOURCE              138.47.163.65
OPEN_PORTS          tcp/22
KEY_BASE64          dGhlIGJpZ2dlc3Qgam9rZSBpbiBjb21wdXRlciBzY2llbmNlIGlz
HMAC_KEY_BASE64     eW91ciBwcm9ncmFtcw==
```

Save and close the file.

3. On the server, delete the copy of the keys.

```
sudo rm /home/spauser/copied-file.conf
```

4. On the server, find the name of the network interface that has your ip address. It is the first column of the line containing your ip address, typically `enp0s3` or `eth0`.

```
ip -br addr
```

5. On the server, open `/etc/fwknop/fwknopd.conf` with `sudo nano` or `sudo vim`. Find the line with `PCAP_INTF`, remove the leading `#`, and replace `eth0` with your interface. If there is no `PCAP_INTF` line, add one. Each line ends with a semicolon:

   ```
   PCAP_INTF                     enp0s3;
   ```

   Save and close the file. If the interface listed here does not exist on your machine, fwknopd will fail to start.

6. On the server, start the fwknop server and check that it is running. The status should show "active (running)".

   ```
   sudo systemctl restart fwknop-server
   sudo systemctl status fwknop-server
   ```

   If it is not running, view the logs with `sudo journalctl -u fwknop-server --no-pager | tail -n 30`.

   On some systems the service is also controlled by `START_DAEMON` in `/etc/default/fwknop-server`. If that file exists and contains `START_DAEMON="no"`, change it to `"yes"` and restart the service again.

> Troubleshooting Tip
>
> If fwknopd will not start or does not seem to see your SPA packets, stop the service and run fwknopd in the foreground with verbose output. It prints every packet it inspects and why it accepts or rejects it. Press `ctrl+c` to stop it, then start the service again.
>
> ```
> sudo systemctl stop fwknop-server
> sudo fwknopd -f -vv -i enp0s3     # replace enp0s3 with your interface
> sudo systemctl start fwknop-server
> ```

4.3 Hiding SSH Behind SPA

1. On the server, set variables for the SSH port and the interface (replace `enp0s3` with your interface).

   ```
   SSH_PORT=22
   INTERFACE=enp0s3
   ```

   Variables only last for the terminal they were set in. If you open a new terminal on the server, set them again.

2. On the server, view the current firewall rules. The INPUT chain should contain one rule that jumps to the `FWKNOP_INPUT` chain, which fwknopd created.

```
sudo iptables -L -nv
```

3. On the server, add a rule that drops all incoming SSH traffic, then view the rules again to see it at the end of the INPUT chain.

```
sudo iptables -A INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
sudo iptables -L -nv
```

4. On the client, try to log in again. It should hang without letting you in. Press `ctrl+c` to give up.

```
ssh -p $SSH_PORT spauser@$SERVER_IP
```

5. On the client, scan the server's SSH port. The STATE should be `filtered`: the DROP rule silently discarded the probe, so the scanner cannot tell whether SSH is even there.

```
sudo nmap -sS -p $SSH_PORT $SERVER_IP
```

6. On the server, open a **second terminal** and watch the fwknop log live. Leave it running.

```
sudo journalctl -u fwknop-server -f
```

7. On the client, send an SPA packet, then **immediately** log in. By default the port is only opened for 30 seconds.

```
fwknop -n $SERVER_IP
ssh -p $SSH_PORT spauser@$SERVER_IP
```

   You should be let in. In the server's log terminal, you should see a message similar to:

   ```
   Added access rule to FWKNOP_INPUT for x.x.x.x -> 0.0.0.0/0 tcp/22,
   expires at 1746380800.
   ```

   If the login hangs, the 30 seconds may have passed. Press `ctrl+c` and repeat this step.

8. On the client, stay logged in and wait until the 30 seconds have passed, then type a few commands. The session will freeze, because fwknopd removed the temporary rule and the DROP rule now applies to your packets. Press `Enter`, then `~` and `.` to force the frozen session to close.

   A real server would add a rule allowing already-established connections so that sessions survive the timeout. This is covered in the full Lab 1.2.

9. **Outsider test.** While the port is open for the client (send a new SPA packet from the client first), try to log in to the server from the outsider machine:

```
ssh -p 22 spauser@138.47.x.x   # the server's ip address
```

   It should hang, since `SOURCE` in `access.conf` only allows the client's ip address, and the temporary rule only matches that ip address. If you have no outsider machine available, skip this step.

10. Stop the log in the server's second terminal with `ctrl+c`.

4.4 SPA Clean Up

Complete these steps before moving on, so the server is not left with a blocked SSH port.

1. On the server, check that the variables are still set (set them again if `echo` prints a blank line), then delete the DROP rule.

```
echo $SSH_PORT $INTERFACE
sudo iptables -D INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
```

2. On the server, stop the fwknop server and prevent it from starting on boot. When fwknopd stops, it removes its own `FWKNOP_INPUT` chain. Confirm the rules from this lab are gone.

```
sudo systemctl stop fwknop-server
sudo systemctl disable fwknop-server
sudo iptables -L -nv
```

3. On the server, open `/etc/fwknop/access.conf` with `sudo nano` or `sudo vim`, change `SOURCE` back to `ANY`, change both key values back to `__CHANGE_ME__`, and delete the `OPEN_PORTS` line.

4. On the server, delete the temporary account.

```
sudo deluser --remove-home spauser
```

5. On the client, delete the saved keys.

```
rm ~/.fwknoprc
```

Note that rules added with `iptables` are not saved, so they are also cleared when a machine reboots. If you ever lock yourself out of a machine while experimenting with firewall rules, restarting it will restore access.

5 Lab: A Simple Chroot Jail

Everyone in the group can do this part on their own machine. You will build a tiny jail containing only `bash` and `ls`, then step inside it.

1. Become the root user, and check your machine's architecture. If it prints `aarch64`, follow the **ARM** commands below. If it prints `x86_64`, follow the **x86-64** commands.

```
sudo -i
uname -m
```

2. Create the jail directory and move into it.

```
mkdir /var/jail
cd /var/jail
```

3. Everything the jailed process needs must be inside the jail. Check which shared libraries `bash` needs:

```
ldd /bin/bash
```

   The output should be similar to one of the following:

```
libtinfo.so.6 => /lib/aarch64-linux-gnu/libtinfo.so.6
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6
/lib/ld-linux-aarch64.so.1
```

```
libtinfo.so.6 => /lib/x86_64-linux-gnu/libtinfo.so.6
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2
```

4. Create the directories and copy in `bash` and its libraries.

```
# ARM Machines:
mkdir bin lib
cp /lib/aarch64-linux-gnu/libtinfo.so.6 lib
cp /lib/aarch64-linux-gnu/libc.so.6 lib
cp /lib/ld-linux-aarch64.so.1 lib
cp /bin/bash bin
```

```
# x86-64 Machines:
mkdir bin lib lib64
cp /lib/x86_64-linux-gnu/libtinfo.so.6 lib
cp /lib/x86_64-linux-gnu/libc.so.6 lib
```

```
cp /lib64/ld-linux-x86-64.so.2 lib64
cp /bin/bash bin
```

5. Add `ls` to the jail. Run `ldd /bin/ls` and copy whatever it lists that is not already in the jail. The commands below show the most common result. If your `ldd` output lists a library not shown here, copy it into `lib` the same way, and if `cp` reports that one of these does not exist, skip it.

```
# ARM Machines:
cp /lib/aarch64-linux-gnu/libselinux.so.1 lib
cp /lib/aarch64-linux-gnu/libpcre2-8.so.0 lib
cp /bin/ls bin
```

```
# x86-64 Machines:
cp /lib/x86_64-linux-gnu/libselinux.so.1 lib
cp /lib/x86_64-linux-gnu/libpcre2-8.so.0 lib
cp /bin/ls bin
```

6. Go to jail.

```
chroot /var/jail
```

7. Explore the jail:

   i. Run `ls /` and `ls /bin`. You should only see what you copied in.

   ii. Run `cd ..` and then `pwd`. You cannot move above `/`, and nothing reveals that you are really in `/var/jail`.

   iii. Try commands that you did not copy in. Each should fail with "command not found", since the programs do not exist inside the jail. (`cd` and `pwd` work because they are built into `bash`.)

```
touch some-file
mkdir some-dir
nano
sudo apt list
```

8. Exit the jail with `exit` or `ctrl+d`. You are back in the normal root shell.

9. Clean up the jail, then exit the root shell.

```
rm -rf /var/jail
exit
```

Make sure you have exited the jail (step 8) before running `rm -rf`. If your prompt still looks unusual, run `exit` once more first.

6 Instructor Demo and Wrap-Up

The simple jail shows the idea, but by itself it does not restrict anyone: you had to be root to enter it, and root can escape a jail. The real access-control use is to jail **ordinary users** automatically when they log in. Your instructor will demonstrate this with OpenSSH, which can lock a group of users into a jail with a few lines of configuration:

```
Match group chrootjail
    ChrootDirectory /var/jail/
    X11Forwarding no
    AllowTcpForwarding no
```

A user in the `chrootjail` group who logs in with `ssh` lands inside the jail and can only use the programs that were copied into it.

Review questions:

1. Which of identification, authentication, authorization, and accountability does an SPA packet provide? Which does the SSH login provide?
2. Why does `nmap` report the SSH port as `filtered` instead of `closed` after the DROP rule is added? Why is that better for the defender?
3. An attacker records your SPA packet with a packet sniffer and sends it again an hour later. What happens, and why?
4. Why must a jailed user never have access to `sudo` inside the jail?

The full **Lab 1.1**, **Lab 1.2**, and **Lab 2** documents contain additional material not covered in class, including keeping SSH sessions open after the SPA timeout, key-based SSH logins, inspecting SPA packets with `tcpdump`, and building a full SSH jail with the `jailsetup.sh` script.